PaxosKV

A complete LeanFM worked example

Project team

November 2, 2026

Contents

1 What this project is **1**
1.1 Reproducing everything . . . 1

2 Requirements **2**
2.1 Boundary and assumptions . . . 2
2.2 Related grammar set . . . 2
2.3 Two-session interaction projection . . . 3
2.4 Observable safety and progress obligations . . . 4
2.5 Prompt-derived outputs . . . 4
2.6 Web behavior . . . 5

3 Derived views **6**

4 Implementation **10**
4.1 Target and processes . . . 10
4.2 Durable state . . . 10
4.3 Consensus and reads . . . 10
4.4 Web application . . . 11
4.5 Resource choices . . . 11
4.6 Observability . . . 11
4.7 Generated source layout . . . 11
4.8 Conformance tests . . . 12
4.9 Runtime . . . 12

5 Running it **13**

6 Two islands over UDP **14**
6.1 The write as a session type . . . 14
6.2 The model the server checks . . . 15
6.3 What the server said . . . 15
6.4 What cannot be observed . . . 18
6.5 Reproducing this chapter . . . 18

7 Known gaps **19**

Chapter 1

What this project is

PaxosKV is a key/value database replicated across three Paxos replicas, with a one-page web front end. Two browser sessions can write at the same time; each write is acknowledged only after two replicas have durably stored it, and every listing shows one committed prefix of the log.

It is the smallest complete example of the LeanFM method: one book (this one), typed requirements in `spec/Requirements.lean` and `spec/Requirements.proto`, an implementation mapping in `spec/Implementation.lean`, Go source in `source/`, views derived from the requirements in `renders/`, and evidence from real runs in `reports/`. Everything the project owns lives under this one directory.

1.1 Reproducing everything

From the project root, with the LeanFM installation built (`make build` two directories up):

```
make spec-check     # compile and validate the Lean specification
make views          # regenerate renders/ from the requirements
make app-check      # go vet, go test -race
make run            # three replicas + web app on 127.0.0.1:8082
make browser-check  # (second terminal) two concurrent browser sessions
make book           # this book, light and dark
```

Replica data lives in `.cache/data/` and survives restarts; `make -C source clean-data` deletes it.

Chapter 2

Requirements

2.1 Boundary and assumptions

The Paxos participant is `KVReplica`, instantiated as one homogeneous cluster of exactly three members: `kv-0`, `kv-1`, and `kv-2`. `Browser` and `WebApp` are actors but are not Paxos participants. Two independent browser sessions may have tasks in flight concurrently.

The confirmed availability premise means that at every modeled instant at least two replicas are running, all replicas can reach one another while running, and each can reach its durable storage. This gives a live majority. Persistence additionally requires an acceptor to fsync its accepted ballot, slot, key, and value before sending `Accepted`; the web response is not successful until a quorum of two durable acceptances exists. A restarted replica recovers its accepted log before participating. If all three replicas stop, the service is unavailable rather than forgetful: after restart, quorum recovery reconstructs the committed log from durable acceptor state before reads or writes resume.

The confirmed conflict rule is last-committed-slot-wins: if the two browser sessions write the same key concurrently, the leader assigns distinct Paxos log slots and the value in the later committed slot is displayed.

2.2 Related grammar set

```
project paxos_kv ::=
  repeat(quorum_write || linearizable_list || recover_replica)

quorum_write ::=
  Browser.Put(key, value, session, request_id) ;
  WebApp.Propose(key, value, session, request_id) ;
  Leader.Assign(ballot, slot) ;
  parallel(
    Replica.Accept(ballot, slot, key, value) ;
    Replica.Accept(ballot, slot, key, value) ;
    Replica.Accept(ballot, slot, key, value)
  ) ;
  mOfN(2, Replica.DurableAccepted(ballot, slot)) ;
  Leader.Commit(ballot, slot, key, value) ;
  WebApp.PutSucceeded(key, value, slot) ;
  ref(linearizable_list)
```

```
linearizable_list ::=
  Browser.List(session, request_id) ;
  WebApp.ReadBarrier(ballot) ;
  mOfN(2, Replica.ReadBarrierAck(ballot, committed_slot)) ;
  Leader.Snapshot(committed_slot, entries) ;
  WebApp.ListSucceeded(committed_slot, entries)

recover_replica ::=
  Replica.Unavailable(instance) ;
  Replica.Restarted(instance) ;
  Replica.RecoverAcceptedLog(instance) ;
  Replica.CatchUp(committed_slot) ;
  Replica.Available(instance)
```

The notation above is intentionally behavioral. An implementation may use a stable Multi-Paxos leader, but leader election and ballot changes must remain observable whenever leadership changes.

2.3 Two-session interaction projection

This is the interaction structure to be generated from the grammar. The two browser branches are incomparable until the leader assigns slots; either may arrive first.



The durable event representation must keep separate `(session, scenario)` keys and use `priorIds` to record immediate predecessor order. `priorIds = []` means that an event is a minimal (first) event in the scenario. If several events have no predecessor, they are mutually unordered and may proceed in parallel. A singleton list is the ordinary prior-event link; a list with several IDs is a join. These links are ordering backpointers, not claims of causation. Event file line order must not serialize otherwise unordered browser requests.

2.4 Observable safety and progress obligations

- A successful PUT necessarily has two prior `DurableAccepted` observations for the same ballot, slot, key, and value.
- No two different values are committed for the same log slot.
- Every committed slot remains present after any one replica is unavailable.
- A list result reflects one committed prefix selected after a quorum read barrier; it does not combine entries from different prefixes.
- With two mutually reachable durable replicas and a leader eventually chosen, every accepted browser request eventually succeeds or returns an explicit error.
- Replica unavailability, leadership loss, retry, rejection, and recovery are visible outcomes rather than hidden timeouts.
- At most three replica instances exist, all implementing the same `KVReplica` actor specification.

2.5 Prompt-derived outputs

`paxos.write.interaction` (interaction diagram) How does a browser write become persistent? *Reducer:* Partition write events by `(session,request_id)` and render quorum branches in `priorIds` order, with typed message fields in each node

`paxos.write.state_machine` (state machine) Which observable write states and failure exits exist? *Reducer:* Render residuals of `quorum_write`

`paxos.list.interaction` (interaction diagram) How is the complete key/value listing made linearizable? *Reducer:* Render list and read-barrier events by `(session,request_id)`

`paxos.list.state_machine` (state machine) Which observable list states and failure exits exist? *Reducer:* Render residuals of `linearizable_list`

`paxos.concurrent.order` (prior-pointer graph) Can two browser sessions access the app concurrently? *Reducer:* Render incomparable request branches, slot assignment, quorum joins, and responses

`paxos.commit_latency` (2D function) How does write latency change with concurrent browser sessions? *Reducer:* Pair PUT start to success/error by prior ID; x=max concurrent sessions, y=latency milliseconds

`paxos.quorum_size` (scalar and histogram) Did every acknowledged write have a durable majority? *Reducer:* Count distinct accepting replica instances per committed slot

`paxos.replica_availability` (line) Were at least two replicas available throughout the observation window? *Reducer:* Sweep `Unavailable`/`Available` intervals and plot available replica count over monotonic time

`paxos.recovery_lag` (2D function) How far behind is a restarting replica? *Reducer:* x=time since restart; y=leader committed slot minus replica applied slot

`paxos.outcomes` (pie/histogram) What happened to browser PUT and list requests? *Reducer:* Count success, explicit rejection, timeout/censored, and leadership-loss outcomes

Numeric reductions use these stable output IDs at `/metrics`. Every diagram, state machine, and chart is derived from the accepted grammar and events rather than maintained as a second specification.

2.6 Web behavior

The page contains a string `key` input, a string `value` input, and a submit action. Below the form it renders the linearizable listing of all committed key/value pairs. After a successful write the initiating session receives a page containing a listing at least as new as its committed write. The other browser session observes that write on its next list operation; server push is not currently required.

There is no authentication and no delete operation. Keys are nonempty UTF-8 strings of at most 128 encoded bytes. Values are UTF-8 strings of at most 4096 encoded bytes. The database holds at most 10,000 distinct keys and supports at least two browser sessions with requests in flight concurrently. Invalid or capacity-exceeding writes return explicit errors without changing the committed log. Each successful submission returns a listing containing at least that committed write.

Chapter 3

Derived views

Every figure in this chapter is generated by `spec/Views.lean` from `Project.Requirements.requirements`; none is drawn by hand.

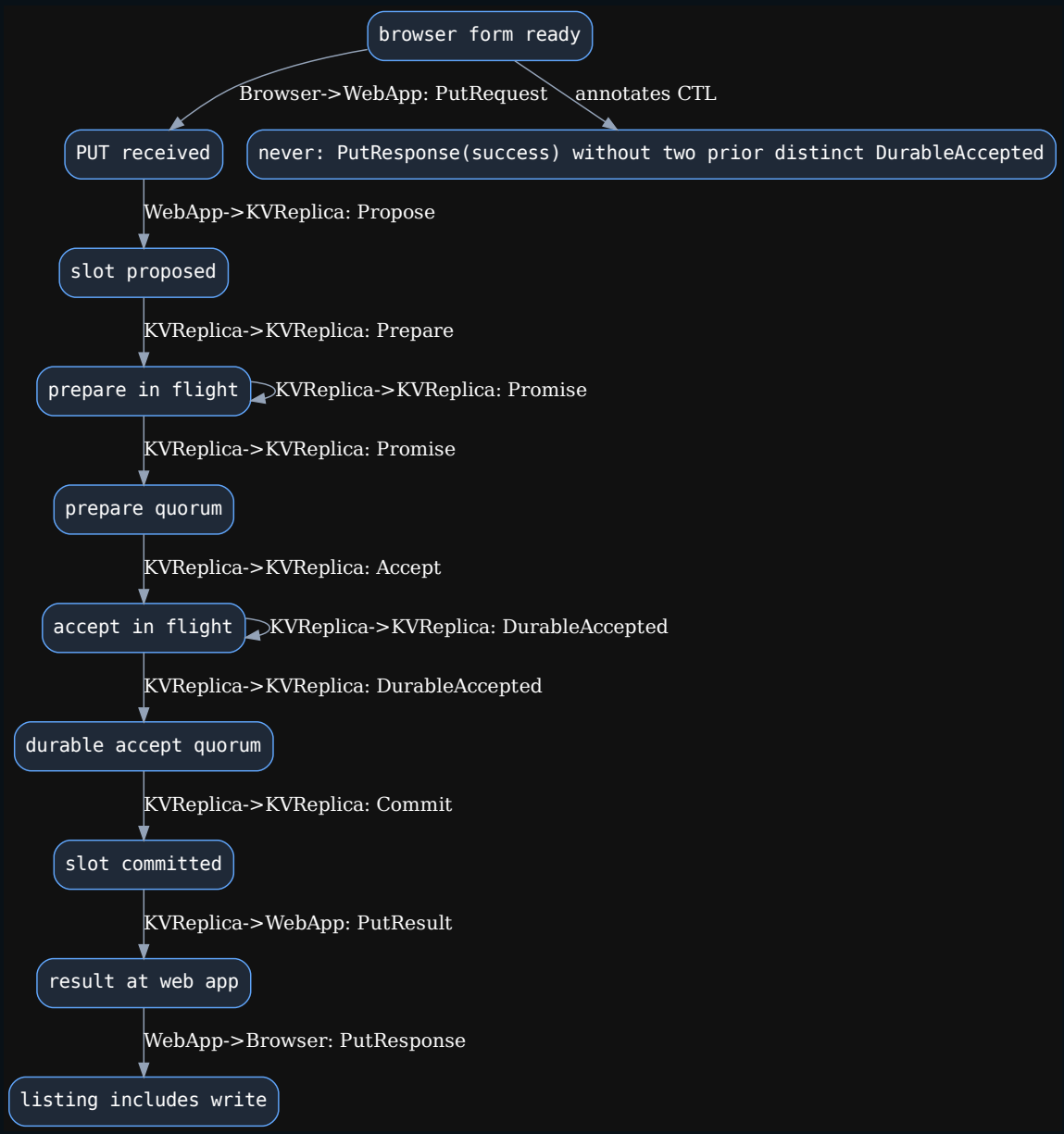


Figure 3.1: Task `quorum_write`: states and transitions.

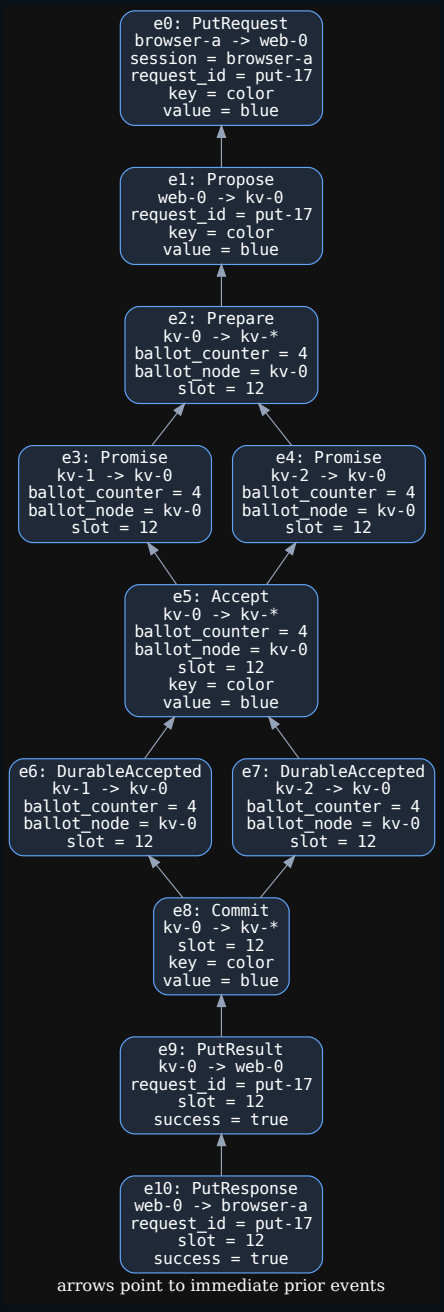


Figure 3.2: One concrete quorum write as a prior-pointer graph. The two `Promise` events fork from `Prepare`; `Accept` and `Commit` are two-of-three joins.

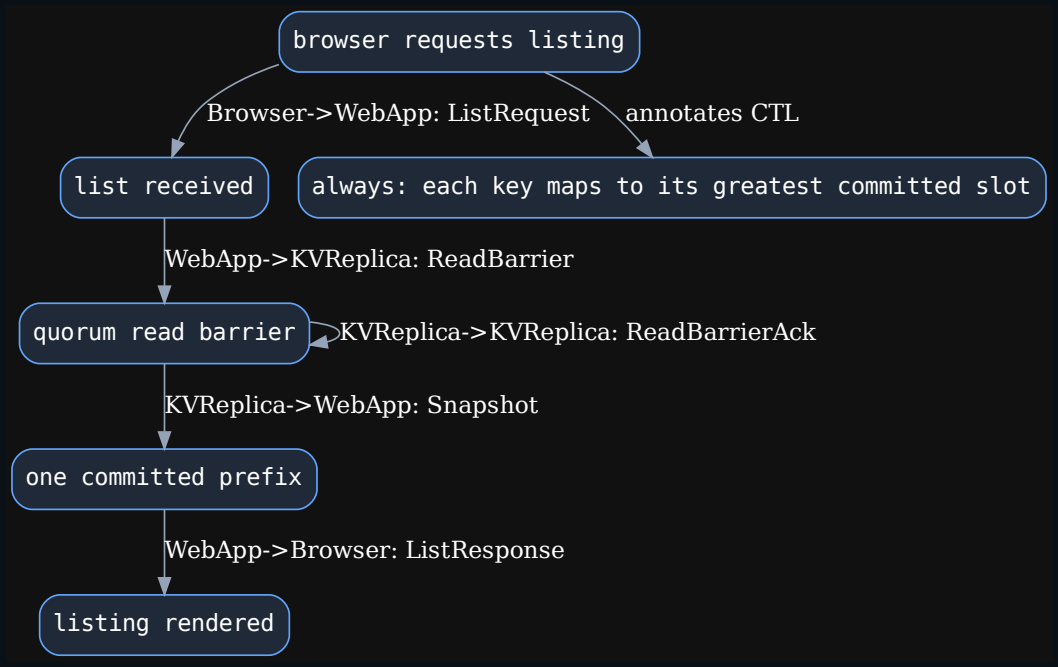


Figure 3.3: Task `linearizable_list`.

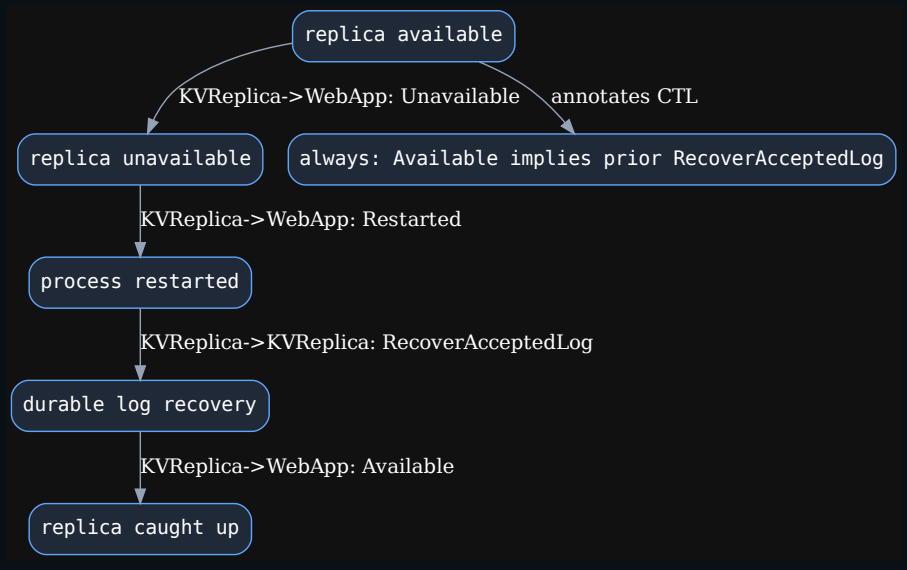


Figure 3.4: Task `recover_replica`.

Chapter 4

Implementation

This chapter maps the requirements to an implementation. It does not change their observable meaning.

4.1 Target and processes

- Target language: Go.
- One `paxos-kv` executable with subcommands `web` and `replica`.
- One WebApp process and three independently started replica processes.
- Replica IDs and addresses are supplied explicitly; membership is fixed at `kv-0`, `kv-1`, and `kv-2`.
- Browser traffic uses HTTP. Replica protocol traffic uses length-delimited protobuf messages over TCP connections.

4.2 Durable state

Each replica owns a separate data directory containing an append-only accepted log and a small metadata file for the highest promised ballot. Every update is written to a temporary record, flushed with `fsync`, and atomically made visible before `DurableAccepted` is sent. Recovery scans and validates the log, truncates only an incomplete final record, and reconstructs accepted slots. No process shares a replica data directory.

The leader records commit decisions durably. On startup or leadership change it runs Paxos recovery against a quorum before accepting HTTP work, choosing the accepted value with the highest ballot for each slot and filling any gaps.

4.3 Consensus and reads

- Stable-leader Multi-Paxos assigns monotonically increasing log slots.
- Ballots are ordered pairs `(counter, replica_id)`.
- A candidate must receive promises from two replicas before becoming leader.
- A value is committed after two distinct replicas have durably accepted the identical `(ballot, slot, key, value)` tuple.
- Repeated messages are idempotent by `(ballot, slot)` and browser `(session_id, request_id)`.

- The materialized key/value map applies committed slots in increasing order; this implements last-committed-slot-wins.
- A list request uses a quorum read barrier and returns one applied committed prefix. A successful PUT waits until its slot is applied, then obtains the listing from a prefix at least that new.

4.4 Web application

- `GET /` obtains a linearizable listing and renders the form followed by the sorted key/value table.
- `POST /put` accepts form fields `key` and `value`, validates encoded byte limits, submits the write, and renders the resulting listing.
- A secure random cookie supplies `session_id`; every request gets a unique `request_id`.
- HTML is escaped. Request bodies, response bodies, and internal queues are bounded. Two browser sessions may progress concurrently.
- Failures return an explicit page and appropriate HTTP status; they never display an uncommitted value as successful.

4.5 Resource choices

- Web request body: 8 KiB maximum.
- WebApp: 64 in-flight requests and a bounded 128-item submission queue.
- Each replica: bounded 256-message inbound queue and 64 simultaneous peer operations.
- Database: 10,000 distinct keys, with limits enforced before proposal.
- Timeouts cancel a client attempt but do not erase a value already accepted; retries use the same request ID and discover the committed result.

4.6 Observability

Every protocol transition is meant to emit the LeanFM event envelope with event ID, repeated predecessor IDs, browser session, task/request ID, concrete source and destination replica IDs, monotonic time, and protobuf atom. `/metrics` exports commit latency, quorum size, available replica count, recovery lag, request outcomes, queue depth, and memory headroom under the accepted output IDs. Logs include ballot and slot but never unescaped HTML.

4.7 Generated source layout

```
source/Makefile                      build, run, stop, test
source/runtime.py                    Runtime actor (supervisor)
source/browser_check.py              two-session browser check
source/cmd/paxos-kv/main.go          replica and web subcommands
source/internal/web/app.go           WebApp and Browser bindings
source/internal/paxos/node.go        KVReplica: Multi-Paxos and recovery
source/internal/paxos/storage.go     fsynced accepted log
source/internal/protocol/            wire Envelope (protobuf)
source/internal/metrics/metrics.go   /metrics
```

`source/internal/trace/events.go`	`events/kv-N.jsonl`
`source/web/templates/index.html`	the page
`source/traceability.json`	requirement -> file map

4.8 Conformance tests

- Two browser sessions write different keys concurrently and both listings converge to the same committed map.
- Two sessions write one key concurrently; the later committed slot wins.
- Every HTTP success has two prior durable acceptances from distinct replicas.
- Stop any one replica during writes; the other two continue and the stopped replica catches up after restart.
- Stop all replicas after acknowledged writes; restart two and recover every acknowledged value before serving.
- Corrupt or truncate the final log record and verify safe recovery without inventing a commit.
- Reject oversized, empty-key, and 10,001st-key writes without mutation.
- Reject conflicting values for one slot, stale ballots, malformed protobuf, and illegal trace order.
- Exercise full queues, timeouts, idempotent retries, and HTML escaping.

The observability section above is the target. Today the replicas emit only terminal events; see the Known gaps chapter.

4.9 Runtime

`source/runtime.py` is the `Runtime` actor: it starts `kv-0`, `kv-1`, `kv-2` on ports 9100–9102 and the web app on 8082, records its own process identity, and stops only that process. Each replica appends its observed events to `events/kv-N.jsonl`.

Chapter 5

Running it

`source/browser_check.py` opens two independent browser sessions against a running deployment. Session A writes `color=blue` while session B writes `shape=circle` concurrently; A then overwrites `color=green`. Both sessions then list the database. The screenshots below are produced by that script; `reports/browser/browser-check.json` records the rows each session saw.

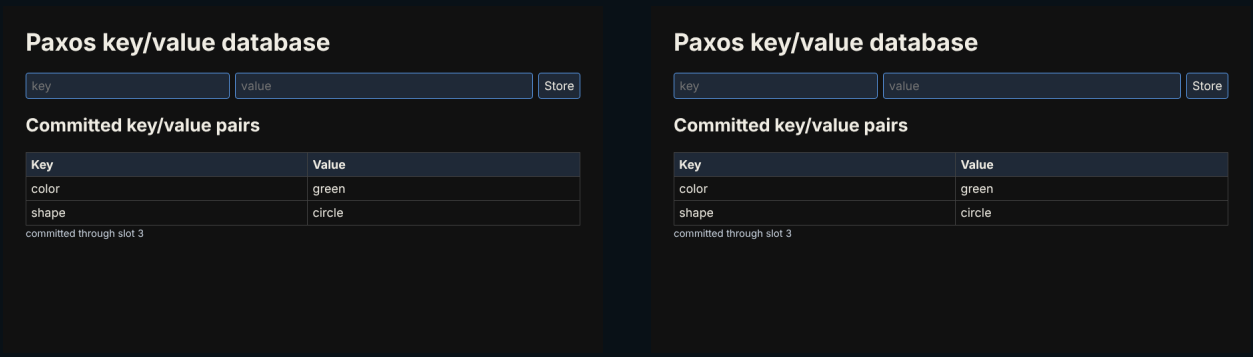


Figure 5.1: Session A (left) and session B (right) after the writes. Both show the same committed map; the later slot for `color` wins.

Chapter 6

Two islands over UDP

PaxosKV's replicas need not share a datacenter. This chapter places `kv0` and `kv1` in island `us` and `kv2` in island `eu`, lets the replicas talk over UDP (as QUIC datagrams would), and asks the LeanFM MCP server what that does to the protocol. Every verdict below is a server answer kept in `reports/mcp/results.json`; the tables and diagrams are typeset from that file by `scripts/mcp_report.py`.

6.1 The write as a session type

`spec/Session.lean` states the write path with the `LeanFM.Session` library. The leader `kv0` already holds a ballot every replica has promised, so a write is phase 2 only: accept on two of three replicas, durably, then answer. The leader's own acceptance is a local fsync, an event at `kv0` alone.

```
protocol QuorumWrite (Browser, Web, Kv0, Kv1, Kv2) {
  put:  PutRequest from Browser to Web;
  prop: Propose from Web to Kv0 cause put where prop.requestId = put.requestId;
  join 2 of {
    Fsync at Kv0;
  } and {
    a1: Accept from Kv0 to Kv1 cause prop;
        DurableAccepted from Kv1 to Kv0 cause a1;
  } and {
    a2: Accept from Kv0 to Kv2 cause prop;
        DurableAccepted from Kv2 to Kv0 cause a2;
  }
  res: PutResult from Kv0 to Web cause prop where res.requestId = prop.requestId;
       PutResponse from Web to Browser cause res;
}
```

The same file declares the physics and the deployment. Browser, web tier and leader talk HTTP; replicas talk UDP to each other with capacity 8. The tree puts the web tier and `kv0` on one machine in `us`, `kv1` on another, and `kv2` alone in `eu`. Its `#guard` lines are checked whenever the file is elaborated:

- the write and the takeover are well formed, and every role can tell which branch it is in;
- every channel's conditions are handled by its protocol: UDP's loss, reordering and duplicates by ballots and retransmission, HTTP's timeouts and retried requests by `requestId`;

- the leader's internal API is open to the web tier and closed to the browser and to `kv2`; its peering port is open to `kv2`;
- a channel from `us` to `eu` surfaces `partition` as well, which the requirements do not yet mention.

The last point is the question for the server: what does a partition, or loss, or reordering across the islands do to one Paxos slot?

6.2 The model the server checks

Most of PaxosKV needs no product at all. A role that forks one thread per session, where each thread ingests only the messages its session expects and sends only the ones it owes, can be checked one session at a time, and that is exact rather than an approximation: no interleaving with another session can change what one thread sees. Two browser writes are such sessions. The leader's ballot and `kv2`'s takeover are not: they are different sessions that meet in each acceptor's promised ballot and accepted value, state that both read and write. That shared state is the only reason to build the product, and it is all the product contains.

`spec/PaxosUdp.lean` builds the product of the three replicas and their UDP channels for one slot. The leader `kv0` proposes `A` at ballot 1. `kv2`, having lost touch with it, takes over with ballot 2: phase 1, then the highest value its two promises report, or its own value `B` if they report none. A sent message may reach each replica in any order, late, or never; a learner reports `ChosenA` or `ChosenB` when two replicas have accepted the same ballot and value. Four variants are written as draft tasks for `check_ctl`:

`correct` as above: 469 states, 1164 transitions.
`broken kv2` ignores what the promises report and proposes `B`: 547 states, 1309 transitions.
`partitioned correct`, with nothing crossing between `us` and `eu`.
`lossy` the leader alone, with every message free to be lost.

Loss needs care. Every prefix of a run that loses messages is also a prefix of a run that only delays them, and the model delays any message arbitrarily, so the safety verdicts hold for lossy UDP as they stand. Liveness does not transfer: in the first three variants every message is eventually delivered, which is the `fairDelivery` assumption written in the session spec. The `lossy` variant drops that assumption on a slice small enough to carry explicit `Lost` steps.

6.3 What the server said

Before checking, each formula was read back by `explain_formula`. The first draft of the agreement property used a plain implication; the server warned that it would also hold if `ChosenA` never happened, so the property uses `stronglyImplies`, as it suggested.

`agreement` (ChosenA was just delivered can happen, and on every path, whenever ChosenA was just delivered then (on every path, always (it is not the case that ChosenB was just delivered))) and (ChosenB was just delivered can happen, and on every path, whenever ChosenB was just delivered then (on every path, always (it is not the case that ChosenA was just delivered))).

`canDecide` on every path, (it is possible that eventually (ChosenA was just delivered or ChosenB was just delivered)) holds unless (ChosenA was just delivered or ChosenB was just delivered) ((ChosenA was just delivered or ChosenB was just delivered) need not happen).

`conflict` it is possible that eventually (ChosenA was just delivered and (it is possible that eventually ChosenB was just delivered)).

`mustDecide` on every path, eventually (ChosenA was just delivered or ChosenB was just delivered).

`possiblyA` it is possible that eventually ChosenA was just delivered.

`possiblyB` it is possible that eventually ChosenB was just delivered.

model	question	verdict	evidence
`broken`	no two different values are both chosen	**disproved**	counterexample: `Accept1A, Accepted1, ChosenA`
`broken`	A is chosen, and later B	**proved**	witness: `Accept1A, Accepted1, ChosenA, Prepare2, Promise0, Accept2B, Accepted0, ChosenB`
`correct`	no two different values are both chosen	**proved**	none, over all 1114 nodes
`correct`	until a value is chosen, one can still be chosen	**proved**	none, over all 1114 nodes
`correct`	A is chosen, and later B	**disproved**	none, over all 1114 nodes
`correct`	a value must be chosen	**proved**	none, over all 1114 nodes
`correct`	A can be chosen	**proved**	witness: `Accept1A, Accepted1, ChosenA`
`correct`	B can be chosen	**proved**	witness: `Prepare2, Promise0, Accept2B, Accepted0, ChosenB`
`lossy`	a value must be chosen	**disproved**	counterexample: `Accept1A, Lost, Lost, then nothing more`
`lossy`	A can be chosen	**proved**	witness: `Accept1A, Accepted1, ChosenA`
`partitioned`	until a value is chosen, one can still be chosen	**proved**	none, over all 11 nodes
`partitioned`	A can be chosen	**proved**	witness: `Accept1A, Accepted1, ChosenA`
`partitioned`	B can be chosen	**disproved**	none, over all 11 nodes

Safety survives the islands. With the correct takeover, no run chooses two values, over all 1114 model nodes, whatever the order in which UDP delivers the messages between `us` and `eu`. Both values are possible on their own: `A` inside `us`, `B` when `kv2` gets in first.

One line of the takeover is the whole difference. With the broken takeover the server returns a run that chooses both. It is short, and it crosses the islands:

The witness came in two questions. `check_ctl` proves $EF(\textit{after ChosenA} \wedge EF\ \textit{after ChosenB})$ with a path that stops where `ChosenA` holds; asking $EF\ \textit{after ChosenB}$ again from that state gives the rest. `check_trace` then accepted the eight events, with their `prevIds` and `causeIds`, and returned the sequence diagram above and the causal graph below. It found nine concurrent pairs: the `us` decision and `kv2`'s prepare are unordered, which is exactly why the takeover must read the promises.

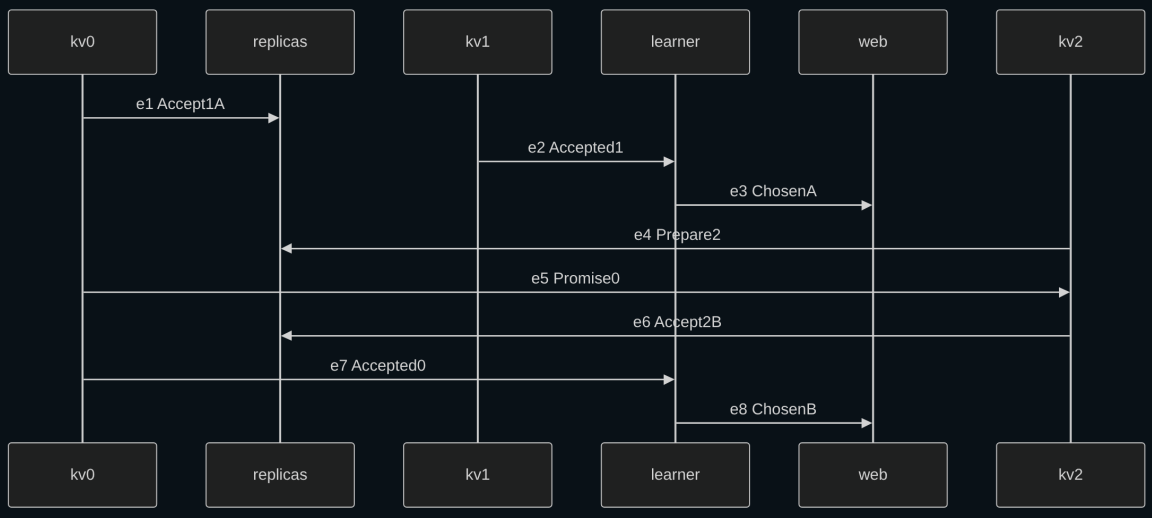


Figure 6.1: The broken takeover, as `check_trace` drew it from the server's witness. `kv0` and `kv1` choose `A` inside `us`. `kv2` then prepares ballot 2; `kv0`'s promise reports that it accepted `A`, `kv2` ignores the report, and `kv0` and `kv2` choose `B`.

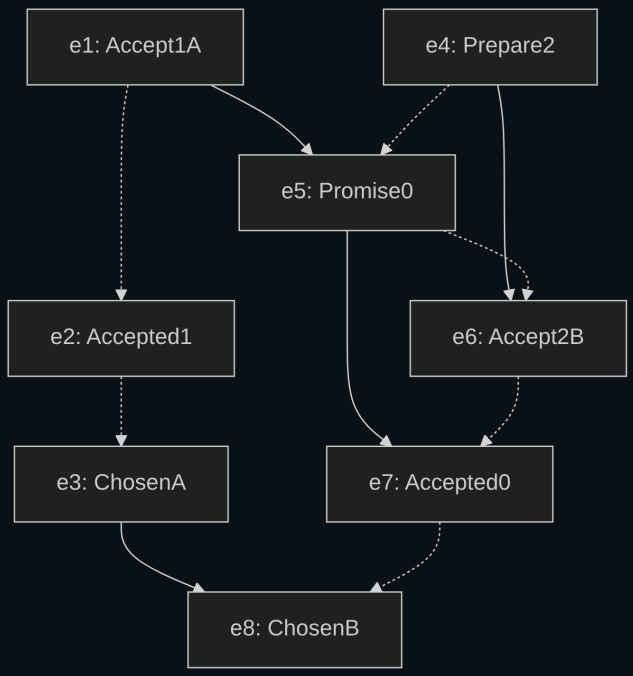


Figure 6.2: Causal graph of the same run. Solid edges are `prevIds` (order at one replica), dashed edges `causeIds`.

A partition leaves a quorum in `us`. Cut `eu` off and `A` can still be chosen; `B` cannot, over all 11 nodes. Until a value is chosen one can always still be chosen, but only on the `us` side: `kv2` alone is a minority.

Liveness needs fairness. Under fair delivery a value must be chosen. Drop fairness and the `lossy` slice returns the obvious lasso: `Accept1A`, `Lost`, `Lost`, then nothing more. No protocol can force UDP to deliver; the promise that a write completes rests on the `fairDelivery` assumption, and on retransmission, which is a timer and therefore physics.

6.4 What cannot be observed

`classify_requirement` sorted three requirement sentences. The first is how agreement is usually stated, and no event can test it; the server proposed an event to expose it. The second restates it over the learner's events, which makes it derived; the agreement formula above is that restatement, and every one of its atoms is observable.

implementation-only the replicas in us and eu agree on the chosen value (proposes event `ReplicasUsEuObserved`

derived ChosenA and ChosenB are never both delivered

implementation-only kv2 has promised ballot 2 (proposes event `Kv2PromisedBallotObserved`)

observable the agreement formula: every atom is observable; check_ctl or check_trace can decide it

6.5 Reproducing this chapter

With the LeanFM server running (`make serve` in the LeanFM root):

```
cd ../..   # LeanFM root
lake build LeanFM.Session
lake env lean projects/PaxosKV/spec/Session.lean          # structural checks
lake env lean --run projects/PaxosKV/spec/PaxosUdp.lean   # draft tasks
cd projects/PaxosKV
python3 scripts/mcp_paxos_udp.py    # ask the server; reports/mcp/results.json
python3 scripts/mcp_report.py       # typeset tables and diagrams (mmdc)
make book
```

Chapter 7

Known gaps

This example runs end to end, but it does not yet close the loop from a running system back to the checker. These are the open items, in the order they should be fixed:

1. The protobuf message types in `spec/Requirements.proto` have no fields; the replicas send one generic `Envelope`. The typed messages in the requirements should become the wire format.

2. Replicas log only terminal events (`PutSucceeded`, `ListSucceeded`) and record no `priorIds`. Every send and receive in the grammar should be logged with its predecessors, so a run can be replayed through `check_trace` and the CTL checker.

3. The interaction diagram for a run should then be drawn from `events/` rather than from the hand-written example scenario.

4. The requirements declare three `WebApp` instances; the runtime starts one.

5. Go tests cover envelope round trips and truncated frames, not message loss, duplication, delay or reordering. The two-island chapter checks these for one slot in a model; the Go replicas still talk over one TCP-style transport and are not exercised under UDP loss.

6. The model in `spec/PaxosUdp.lean` encodes the replicas' rules by hand; it is not yet derived from the session type in `spec/Session.lean`. It is shaped to fit the MCP server's bounds (64 KiB of arguments, 4096 nodes): one slot, two ballots, no duplicate delivery, and loss only in the leader-only slice.